DOCUMENT OF RECORD

Kickoff prompt

Project	Agent-Team-Built Enterprise Azure DevSecOps Demo
Sponsor	Paul Fuqua
Issued	2026-08-22
Status	Founding brief — authoritative; amendments recorded separately

> Reproduced as issued. The brief sections below are also committed at `docs/BRIEF.md`; this document additionally carries the opening session instruction and the closing first task, which that file omits. The stack decision naming the LLM provider is left exactly as written — it was superseded on 2026-08-24, and the amendment lives in `docs/superpowers/specs/` rather than being edited into this record.

> Enable Agent Teams for this session (`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`). Use the Superpowers workflow: brainstorm to pressure-test this brief, produce a written implementation plan, then execute via the agent team topology below. Treat this document as the project brief and the record of decisions already made.

Mission

Build a fully agent-instantiated demo of an enterprise-grade Azure environment, themed for the space launch industry, that can be destroyed and rebuilt on demand to control costs. The repo is the product; the Azure environment is a build artifact. Everything is authored as code/scripts and instantiated via pipelines, so the entire environment can be recreated from nothing.

Audience: launch-industry engineering/security/ops leaders. The demo must read as enterprise-real, not hobby-grade.

Agent team topology

Lead pair (persistent, mutual accountability):

- **Orchestrator** — owns the build order and dependency graph, decomposes layers into tasks on the shared task list, assigns workstream leads, and is the single channel for human escalation. Restarts stalled teammates.
- **Verifier** — independent state auditor. After each layer, queries Azure/Graph/Fabric/GitHub APIs directly (never trusts a teammate's self-report) and confirms deployed state matches intent. A layer is DONE only on Verifier sign-off. Also monitors the Orchestrator: if it stalls or drifts from this brief, the Verifier flags and restarts it.

Workstream leads (report to Orchestrator, message each other directly for cross-cutting dependencies):

1. **Platform Lead** — Bicep/AVM scaffolding, landing zone, Container Apps environment, Azure SQL serverless, Log Analytics/App Insights, cost exports, GitHub OIDC, CI/CD pipelines, teardown/reinstantiate scripts.
2. **Identity & Governance Lead** — Entra users/groups/CA policies/app registrations (Graph PowerShell), Purview labels, Azure Policy assignments, tagging enforcement, NIST 800-53 initiative.
3. **Data & Copilot Lead** — Fabric workspace/lakehouse (REST API), synthetic launch-industry data generators and seeding, copilot service (SQL-over-lakehouse tool use, JSON component-spec output), shared Fluent UI spec-renderer library.
4. **DevSecOps Lead** — GHAS configuration (Dependabot, CodeQL, secret scanning), SBOM (SPDX via sbom-tool/Syft), Trivy in CI, ZAP DAST stage, Defender toggle scripts, self-healing pipeline, control tower data feeds.

IC agents: each lead spawns 2–5 ICs (specialists or generalists as the task demands: Bicep author, Fluent UI builder, data generator, PowerShell author, pipeline author, etc.).

Rules of engagement:

- Team messaging (SendMessage) exists only at the lead/teammate tier. Disposable exploration sub-agents get no messaging access — leads must not become message routers for grep output.
- Each teammate works in its own git worktree; leads merge via PR to main.
- ICs may run autonomously for extended stretches on repo work (authoring IaC, apps, scripts, tests, docs). Autonomy applies to authoring, never to deployment.
- All escalations flow: IC → workstream lead → Orchestrator → human. Only the Orchestrator messages the human, except the Verifier, which may escalate directly if the Orchestrator itself is the problem.

Human-in-the-loop gates (deliberately minimal)

A real A&D enterprise would gate heavily; this demo gates only where money, credentials, or irreversibility are involved. The Verifier's independent state audit replaces routine human review everywhere else. Exactly five gates:

1. **G0 — Bootstrap (one-time, human-only):** tenant + subscription with billing; root OIDC federation/service principal with Owner + admin-consented Graph and Security & Compliance permissions; Fabric F2 capacity purchase. Agents verify preconditions at session start and emit a precise checklist if anything is missing — they never attempt these.
2. **G1 — Master plan approval (once):** Orchestrator presents the full plan (repo structure, layer sequence, per-layer verification criteria, projected cost envelope). Human approves once; thereafter layers auto-apply with Verifier sign-off substituting for human review. No per-layer approval.
3. **G2 — Spend-profile changes (each occurrence):** anything that raises the run-rate — Fabric capacity resume, Defender for Containers enable, SKU changes, any resource without scale-to-zero/auto-pause. Orchestrator states projected cost delta and duration; human approves.

4. **G3 — Tenant-level destructive operations (each occurrence):** deletion of Entra objects, Purview labels, Fabric workspace, or the OIDC federation. RG-scoped teardown of demo resources requires no approval — that path must stay frictionless for the kill/reinstantiate demo.
5. **G4 — Exception escalation (event-driven):** Verifier fails a layer twice, cost anomaly detected, credential/permission failure, or lead-pair deadlock. Otherwise the human is not in the loop.

Self-healing pipeline note: within the demo environment, auto-merge on green is intentionally allowed (that IS the showpiece). The human sees the PR trail, not an approval prompt.

Non-negotiable principles

1. **Repo as source of truth.** No manual portal configuration outside G0. Anything created must be re-creatable by re-running pipelines.
2. **Kill/reinstantiate in under an hour.** Teardown = delete demo RGs + scripted cleanup of tenant-level objects. Rebuild = replay pipelines + seed scripts. Idle cost near zero (serverless/scale-to-zero everywhere, Fabric paused). Every layer ships its teardown script alongside its create path.
3. **Layer-by-layer with Verifier gates.** Deploy, independently audit actual state via APIs, then unblock the next layer. No monolithic apply — Entra propagation and policy assignment lag will break it.
4. **Synthetic data only.** Launch-industry-flavored synthetic data (launches, scrubs, providers, telemetry, parts, supply chain) with realistic messiness. Public facts (vehicle names, launch sites) are fine. Nothing proprietary from any employer.
5. **Microsoft-native and standards-based.** Azure Verified Modules, Cloud Adoption Framework landing-zone patterns, Well-Architected framing, Azure Policy governance, SPDX SBOMs, OpenTelemetry, OIDC with no stored cloud secrets in CI.

Stack decisions (already made — do not relitigate without strong cause)

IaC: Bicep + AVM for all Azure resources. Graph PowerShell for Entra objects. Security & Compliance PowerShell for Purview labels. Fabric REST API scripts for workspace/lakehouse. Teardown is RG-scoped deletion plus cleanup scripts for tenant-level objects.

Landing zone (mini): management groups, Azure Policy assignments enforcing tagging and guardrails, NIST 800-53 initiative assigned to the demo subscription.

Identity: Entra ID users, groups, CA policies, app registrations, service principals. GitHub Actions → Azure via OIDC federation.

Data: Fabric OneLake Lakehouse (F2, paused when idle) as the cross-domain analytical plane — all apps' data lands or syncs there. Azure SQL Serverless (auto-pause) as operational DB per app. Purview sensitivity labels (persist across teardowns).

Compute: One Container Apps environment hosting all apps + the copilot service, all scale-to-zero. Functions (consumption) for small glue.

Frontend: React + Fluent UI v9 as container apps, sharing a JSON-spec renderer library (~8–10 components: bar/line charts, stat cards, tables, timelines).

Copilot service (showpiece #1): LLM-backed service (Anthropic API or Azure AI Foundry) with tools over the Fabric lakehouse SQL analytics endpoint plus the ops/sec/cost APIs. Answers cross-domain natural-language questions (e.g., "which day of the week has the most launches and which day of the year has the most scrubs") by generating SQL, executing it, and returning a JSON component spec — never runtime-generated React. The fixed renderer draws the answer.

Control tower app (showpiece #2): Dev / Sec / Ops tabs framed on Well-Architected pillars. Dev: vulns, dependency status, SBOM, PR/pipeline status. Sec: Defender secure score, findings by severity, NIST posture, Entra sign-in risk. Ops: resource health, throughput/latency, replica counts, cost-over-time. Sources: cost exports → lakehouse, Log Analytics KQL API, App Insights, GitHub Security APIs, Defender APIs. The copilot shares these tools.

DevSecOps chain: GitHub repos with Advanced Security (public repos for free tier). SPDX SBOMs; optionally Dependency-Track as a container app. Trivy in CI; Defender for Containers at runtime (toggleable, G2-gated). ZAP DAST against staging. OpenTelemetry → App Insights.

Self-healing code (showpiece #3): Dependabot/CodeQL finding → agent triage → PR with patch and explanation → CI gauntlet (SAST, tests, ZAP) → auto-merge on green → deploy → finding closed. GitHub Actions + Claude API. Seed intentionally vulnerable dependencies so it has something to heal.

FinOps: Policy-enforced tag taxonomy; daily cost exports to the lakehouse; cost-per-app-over-time as a first-class control tower visual.

Build order

Each layer: assigned lead → ICs author → deploy → Verifier audit → unblock next.

1. Repo skeleton, tagging taxonomy, Bicep/AVM scaffolding, OIDC wiring, infra up/down pipelines — Platform
2. Landing zone: management groups, policies, NIST initiative — Identity & Governance
3. Entra layer: users, groups, CA policies, app registrations — Identity & Governance
4. Purview labels — Identity & Governance
5. Fabric workspace, lakehouse, data generators, seeding — Data & Copilot
6. Core platform: Container Apps env, SQL serverless, Log Analytics, App Insights, cost exports — Platform
7. Apps: launch-data app + control tower with shared spec renderer, per-app CI/CD — Data & Copilot + Platform
8. Copilot service with lakehouse + ops/sec/cost tools — Data & Copilot
9. DevSecOps chain: GHAS, SBOM, Trivy, ZAP, Defender toggles — DevSecOps
10. Self-healing workflow with seeded vulnerabilities — DevSecOps
11. Teardown/reinstantiate proof: destroy and rebuild end to end — Platform, Verifier-audited

First task for this session

Orchestrator: verify G0 preconditions (emit checklist if incomplete), then assemble the team, and present the G1 master plan — repo structure, layer plan with per-layer verification criteria, and projected cost envelope — for my one-time approval. **Write nothing to Azure before G1.**

Meridian Launch Systems is a fictional launch provider introduced during execution of this brief. All data in the resulting environment is synthetic.